PWM with Timer
Nathaniel Ross

Task A: PWM Timer Init

Do: Initialize the PWM timer and show it works by wiring an LED to the STM32

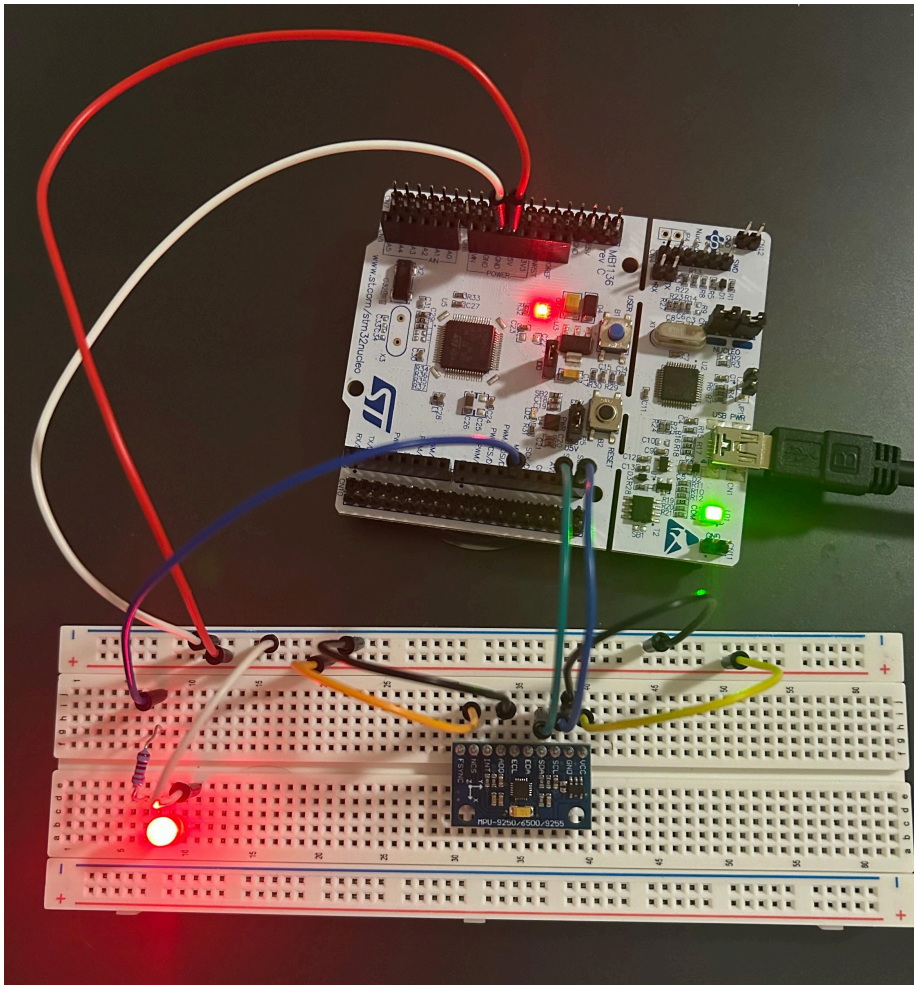
Code:

tim3_pwm_init.c:

```
#include "tim3_pwm_init.h"

void tm3_init(void)
{
	// 1. peripheral clock enables
	RCC->AHB1ENR |= 1;			// GPIOA
	RCC->APB1ENR |= (1 << 1);	// TIM3

	// 2. configure GPIOA as alternate function
	GPIOA->MODER &= ~(0b11 << 12);
	GPIOA->MODER |= (0b10 << 12);  // AF mode

	// set to push-pull
	GPIOA->OTYPER &= ~(1 << 6);

	// set to no pull-up pull-down
	GPIOA->PUPDR &= ~(0b11 << 12);

	// configure AF02 for PA6
	GPIOA->AFR[0] &= ~(0xF << 24);
	GPIOA->AFR[0] |= (0x2 << 24);

	// 3. configure timer base
	TIM3->PSC &= ~(0xFFFF);
	TIM3->PSC |= (0b101000111);    // set PSC to 327
	TIM3->ARR &= ~(0xFFFF);
	TIM3->ARR |= (0xFF);           // set ARR to 255

	// 4. configure PWM channel
	TIM3->CCMR1 &= ~(0b111 << 4);
	TIM3->CCMR1 |= (0b110 << 4);   // PWM Mode 1
	TIM3->CCMR1 |= (1 << 3);       // enable preload register
	TIM3->CCR1 &= ~(0xFFFF);
	TIM3->CCR1 |= (0x80);          // CCR1 = 128, ~50% duty cycle

	// 5. enable output channel
	TIM3->CCER &= ~(0b11);
	TIM3->CCER |= (0b01);          // enable channel active high
```

```
        // 6. enable auto-reload preload
        TIM3->CR1 |= (1 << 7);

        // 7. force update for PSC/ARR/preload values
        TIM3->EGR |= 0b1;  // set UG (update generation)

        // 8. start timer
        TIM3->CR1 |= 0b1;  // set CEN (counter enable)
}
```

Results:



LED (bottom left) is lit up to about 50% brightness, as intended

Learned:

- How to generate a PWM signal on the STM32F411RE using the TIM3 peripheral and bare-metal programming
- Choose to use PA6 (D12 on board) for output to LED. This meant TIM3-CH1 was the timer and AF02 was the alternate function used based on the alternate function mapping table in the STM32F411RE datasheet
- A timer counts from 0 to ARR and compares the counter value with CCR1, generating the PWM
- PWM frequency determined by timer clock, prescaler (PSC), and auto-reload register (ARR / period): f_PWM = f_CLK / ((PSC+1)(ARR+1))
- There is a tradeoff between frequency accuracy and duty-cycle resolution: My PWM frequency was not able to hit exactly 1kHz, however I accepted that slight inaccuracy instead of raising the resolution
- Duty cycle is controlled by CCR1. In PWM Mode 1, raising CCR1 means higher duty cycle, while PWM Mode 2 is opposite
- Placing CCR1 into shadow register ensures duty cycle changes only occurs when PWM finishes period. ARPE enabled ensures ARR does not change before period.
- Setting the UG bit ensured that PSC, ARR, CCR1 were updated during initialization
- In general, I gained more experience reading STM32 reference manuals and datasheets to determine which bits, flags, and registers to manipulate. In turn, this has given me more understanding in embedded functionality by learning the purpose of each register involved during initialization

Task B: UART Command

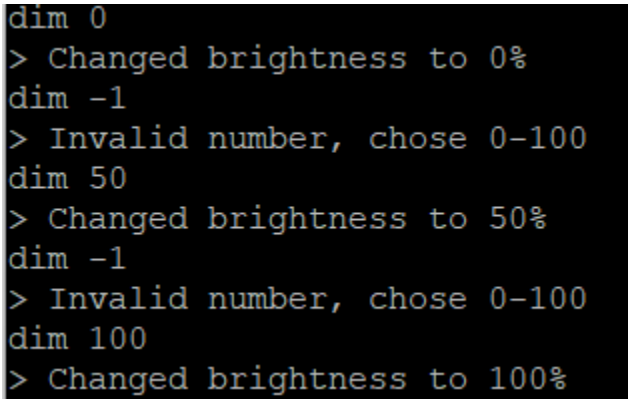
Do: Add a command to the UART shell to change the LED's brightness by manipulating its duty cycle.

Code:

main.c:

```
int brightness;
// change attached LED brightness
else if(sscanf(cmd, "dim %d", &brightness) == 1)
{
	if(brightness >= 0 && brightness <= 100)
	{
		uint8_t val = brightness * 255 / 100;
		TIM3->CCR1 = val;

		char msg[64];
		snprintf(msg, sizeof(msg), "> Changed brightness to %d%%\r\n",
brightness);
		HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
	}
	else
	{
		char msg[64];
		snprintf(msg, sizeof(msg), "> Invalid number, chose 0-100\r\n");
		HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
	}
}
```

Results:



Learned:

- How to easily change duty cycle to change LED brightness with TIM3->CCR1
- Scale brightness level to correct value based on 0-255 range
- Parse an input command with sscanf

Task C: Stream data with interrupts

Do: Fix the “stream” command from the previous project to be non-blocking by using interrupts.

Code:

tim_init.c:

```
volatile uint8_t data_ready;
void tim2_init(void)
{
	RCC->APB1ENR |= (1 << 0);		// TIM2
	TIM2->CR1 = 0;				// reset timer configuration
	TIM2->PSC = 1281;			// PSC = 1281
	TIM2->ARR = 131044;			// 65522 = 1 second
	TIM2->EGR |= 1;				// set update generation
	TIM2->SR &= ~0b1;			// clear UIF
	TIM2->DIER |= 0b1;			// update interrupt enable
	TIM2->CR1 |= (1 << 7) | 1;		// ARPE, CEN
	NVIC->ISER[0] |= (1 << 28);		// interrupt controller enable
}
void TIM2_IRQHandler(void)
{
	TIM2->SR &= ~0b1;	// clear UIF
	data_ready = 1;		// signal timer period finished
}
```

main.c:

```
// non-blocking for streaming MPU sensor data
if(streaming && data_ready)
{
	print_mpu_data();
	data_ready = 0;
}
```

Results:

```
stream
> Streaming MPU Data
> Accel X: 0.10
> Accel Y: 0.09
> Accel Z: 1.04
> Temp (C): 22.96
> Gyro X: -3.66
> Gyro Y: 9.21
> Gyro Z: -1.49
> Accel X: 0.11
> Accel Y: 0.09
> Accel Z: 1.03
> Temp (C): 22.87
> Gyro X: -3.60
> Gyro Y: 9.51
> Gyro Z: -1.49
stop
> Stopping Data Stream
```

Learned:

- How to use an interrupt timer to read sensor data without polling
- How to manipulate the prescaler (PSC) and auto-reload (ARR) to have the sensor data read once every *n* seconds (set the period to n seconds)
- The “extern” key allows for variables to be used between files. When using the “data_ready” state variable, it was declared with the extern key in main.c to recognize it is also used in tim_init.c
- The update interrupt flag (UIF) should be cleared at the start of the interrupt handler instead of the end because it prevents new interrupts from being erased
- The timer has to be configured for interrupts with TIMx->DIER and NVIC has to be configured to allow an interrupt from that timer with NVIC->ISER. The specific bit in NVIC->ISER to be enabled for TIM2 is found in the stm32f411xe.h file
- Similar to the callback function for the UART interrupt communication, a timer uses an IRQHandler function to specify what to do once the timer period has elapsed

Task D: LED Pulse

Do: Add a pulse command that will use timer interrupts to smoothly pulse the LED until told to stop

Code:

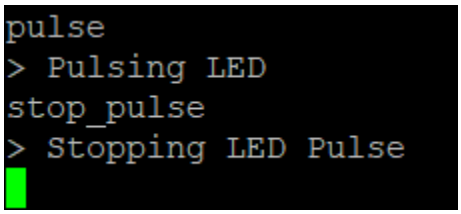
main.c:

```
main() while loop
if(pulse_state != 0 && pulse_ready)
{
	// pulse based on state
	if(pulse_state == 1)
		pwm_val++;
	else if(pulse_state == 2)
		pwm_val--;

	// change state
	if(pwm_val == 0)
		pulse_state = 1;
	else if(pwm_val == 128)   // only lights up to 50% brightness
		pulse_state = 2;

	TIM3->CCR1 = pwm_val;
	pulse_ready = 0;
}

process_command() function
else if(strcmp((char*)cmd, "pulse") == 0)
{
	pwm_val = 0;
	TIM3->CCR1 = 0;
	pulse_state = 1;
	char msg[64];
	snprintf(msg, sizeof(msg), "> Pulsing LED\r\n");
	HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
else if (strcmp((char*)cmd, "stop_pulse") == 0)
{
	pulse_state = 0;
	char msg[64];
	snprintf(msg, sizeof(msg), "> Stopping LED Pulse\r\n");
	HAL_UART_Transmit(&huart2, msg, strlen(msg), HAL_MAX_DELAY);
}
```

tim_init.c:

```
void tim4_init(void)
{
	RCC->APB1ENR |= (1 << 2);		// TIM 4 peripheral clock
	TIM4->CR1 = 0;
	TIM4->PSC = 1281;
	TIM4->ARR = (PULSE_INTERVAL_SEC * TICKS_PER_SECOND);
	TIM4->EGR |= 1;
	TIM4->SR &= ~0b1;
	TIM4->DIER |= 0b1;
	TIM4->CR1 |= (1 << 7) | 1;
	NVIC->ISER[0] |= (1 << 30);
}
void TIM4_IRQHandler(void)
{
	TIM4->SR &= ~0b1;
	pulse_ready = 1;
}
```

Results:



LED starts at 0, slowly lights up, and then slowly dims back to 0 repeatedly until stop_pulse ran

Learned:

- How to use a timer to pulse an LED. This command can also run concurrently with the “stream” command, meaning the LED will pulse and the MPU-6500 sensor data will be read to the console at the same time.
- How to create a state-machine for LED state: 0 = On, 1 = Brighten, 2 = Dim
- Reinforced concepts learned in the previous task (streaming data with timer interrupts), including the use of the “extern” key, clearing the UIF flag at the start of the interrupt handler, configuring the timer for interrupts, configuring NVIC for interrupts from that timer, and writing an IRQHandler

Task E: Refactor Code

Do: Refactor the project code to be organized and easy to follow. It is currently a jumbled mess.

Results:
Turned the process_command() function into its own process_command.c file with separate functions for each command:

```c
void process_command(uint8_t* cmd)
{
    uint8_t newline[] = "\r\n";
    HAL_UART_Transmit(&huart2, newline, 2, HAL_MAX_DELAY);
    int brightness;

    if(strcmp((char*)cmd, "led on") == 0)               // turns led on
        cmd_led_on();

    else if(strcmp((char*)cmd, "led off") == 0)         // turns led off
        cmd_led_off();

    else if(strcmp((char*)cmd, "status") == 0)          // fetches led status
        cmd_status();

    else if(strcmp((char*)cmd, "whoami") == 0)          // returns MPU's identity
        cmd_whoami();

    else if(strcmp((char*)cmd, "read") == 0)            // reads accel, temp, and gyro data from MPU
        print_mpu_data();

    else if(strcmp((char*)cmd, "stream") == 0)          // constantly streams MPU sensor data
        cmd_stream();

    else if(strcmp((char*)cmd, "stop") == 0)            // stops stream of MPU sensor data
        cmd_stop();

    else if(sscanf(cmd, "dim %d", &brightness) == 1)    // change attached LED brightness
        cmd_dim(brightness);

    else if(strcmp((char*)cmd, "pulse") == 0)           // sets LED state to pulse
        cmd_pulse();

    else if (strcmp((char*)cmd, "stop_pulse") == 0)     // sets LED state to normal
        cmd_stop_pulse();

    else cmd_unknown();                                 // unknown command entered
}
```

Completely cleaned up main.c while() loop:

```c
/* USER CODE BEGIN WHILE */
while (1)
{
  handle_uart_cmd(cmd_buff, &cmd_index); // for user inputs
  stream_mpu();   // streams MPU sensor data
  pulse_led();    // pulsing command

  /* USER CODE END WHILE */

  /* USER CODE BEGIN 3 */
}
```

Learned:

- How to refactor code to make it easier to use, follow, and show intended functionality
- Use the extern keyword to allow functions to be used between files

What was learned overall:

- How to generate a PWM signal using timers and bare-metal programming
- Alternate function GPIO mode has to configured for PWM timers
- Timers work by counting from 0 to ARR, and comparing with CCR1. CCR1 controls the duty cycle by saying how much time the signal is high before going low
- PWM frequency follows the equation:
    - f_PWM = f_CLK / ((PSC+1)(ARR+1))
    - Inverting this can be used to find the period
- Timers can also be used for non-PWM functions, such as displaying sensor data. With this, PSC and ARR can be manipulated to have the sensor data read once every n seconds.
- A timer has to be configured for interrupts with DIER register, and the NVIC (interrupt controller) has to be configured for interrupts from that timer with the ISER register
- A timer uses an IRQHandler function to specify what to do once a timer period has elapsed, similar to the UART callback function
- The “extern” keyword allows for functions and variables to be used between different files
- Interrupts allow for multiple commands to be run concurrently: LED continuously pulses while the MPU-6500 continuously reads data
- How to refactor code to be easier to read, follow, and show intended functionality
- This project can still be improved in many ways. More sensors, devices, and commands could be added, but importantly more communication protocols could be used. In the future, a device that uses the SPI protocol could be added so that the project utilizes UART, I2C, and SPI protocols.